

Production Supporting Systems in Factories

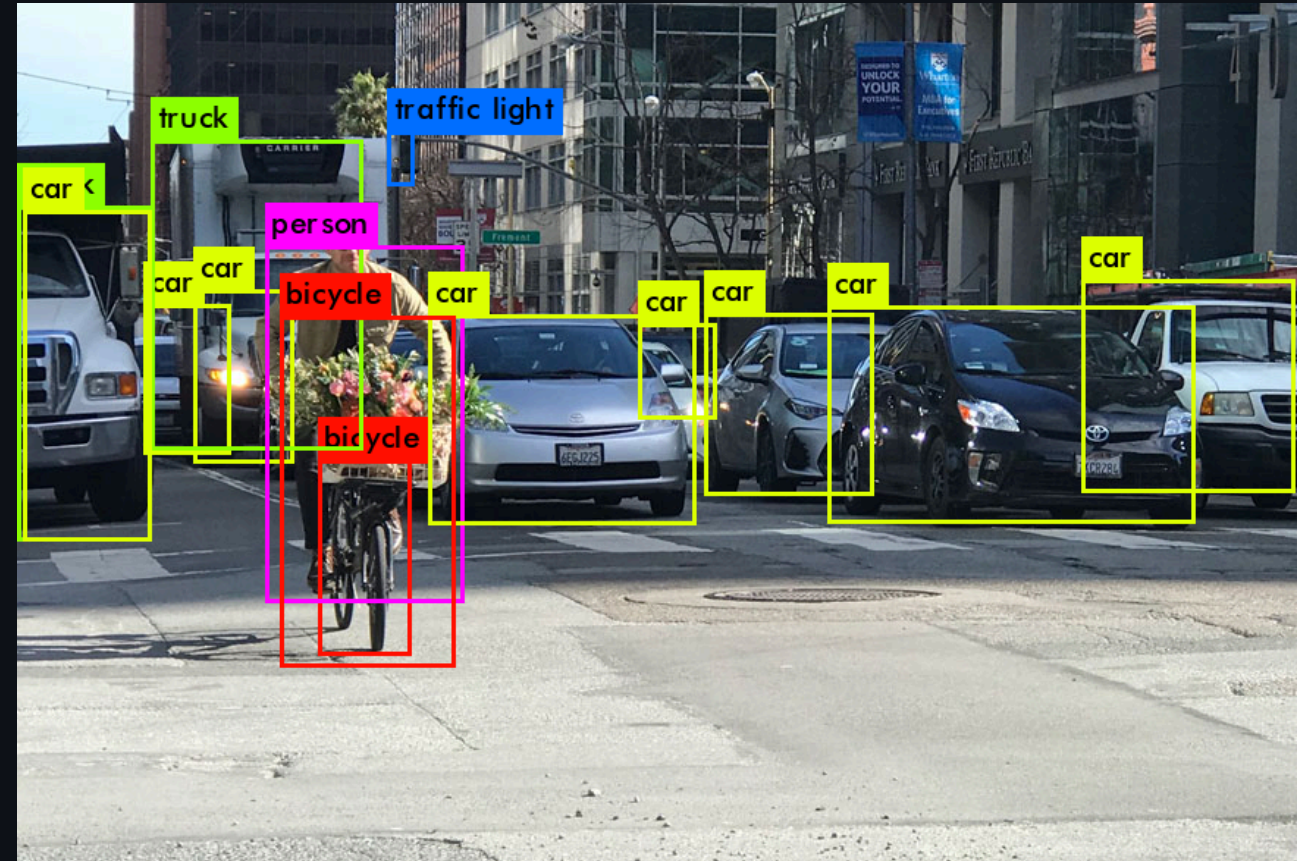
ระบบสนับสนุนการผลิตในโรงงานอุตสาหกรรม

Machine Learning

| Object Detection

Object Detection

- Car
 - Top: 500, Bottom: 200, Left: 50, right: 400
 - 50%
- Bicycle
 - ...
 - ...



COCO Dataset

- *Common Objects in Context*
- Large-scale image recognition dataset for object detection, segmentation, and captioning tasks.
 - Contains over 330,000 images.
 - Annotated with 80 object categories.
- <https://cocodataset.org/#explore>

DETR Model

- DEtection TRansformer (DETR) model by Facebook AI.
- Uses transformer architecture for object detection.
- Pre-trained on COCO dataset.
- Classes

Setting Up ML Server

- Get [code](#)
- `pnpm install`
- `pnpm run build`
- `pnpm run start`
- Test if server is running.
 - <http://localhost:3004>

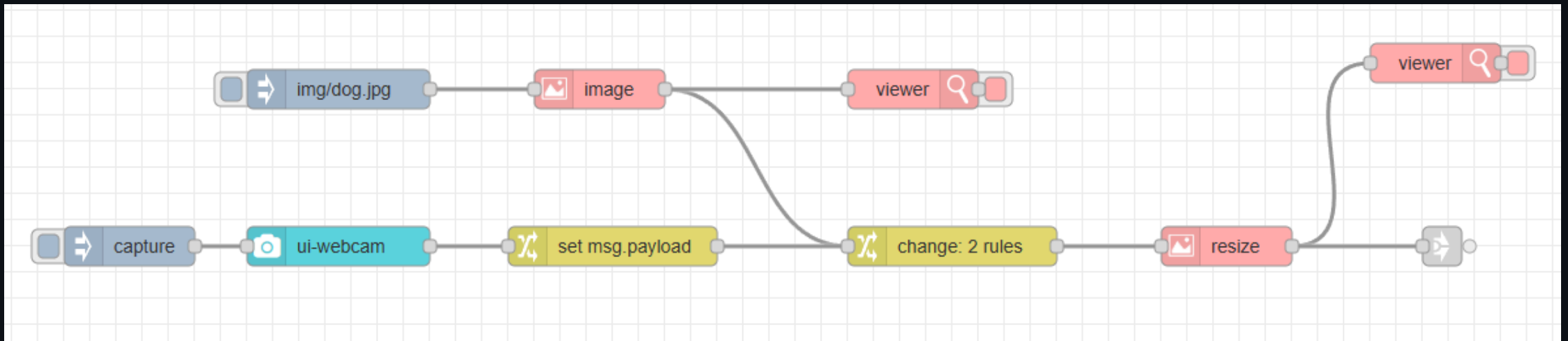
Install Additional Modules

- Go to Menu (top right) -> Manage palette -> Install
- Install the following modules:
 - `node-red-contrib-image-tools`
 - `@sumit_shinde_84/node-red-dashboard-2-ui-webcam`

Prepare Mock Images

- Copy image from `ml-express/temp/img` to your node-red installation directory under `img` folder.

Image Input Flow



Inject Node Configuration

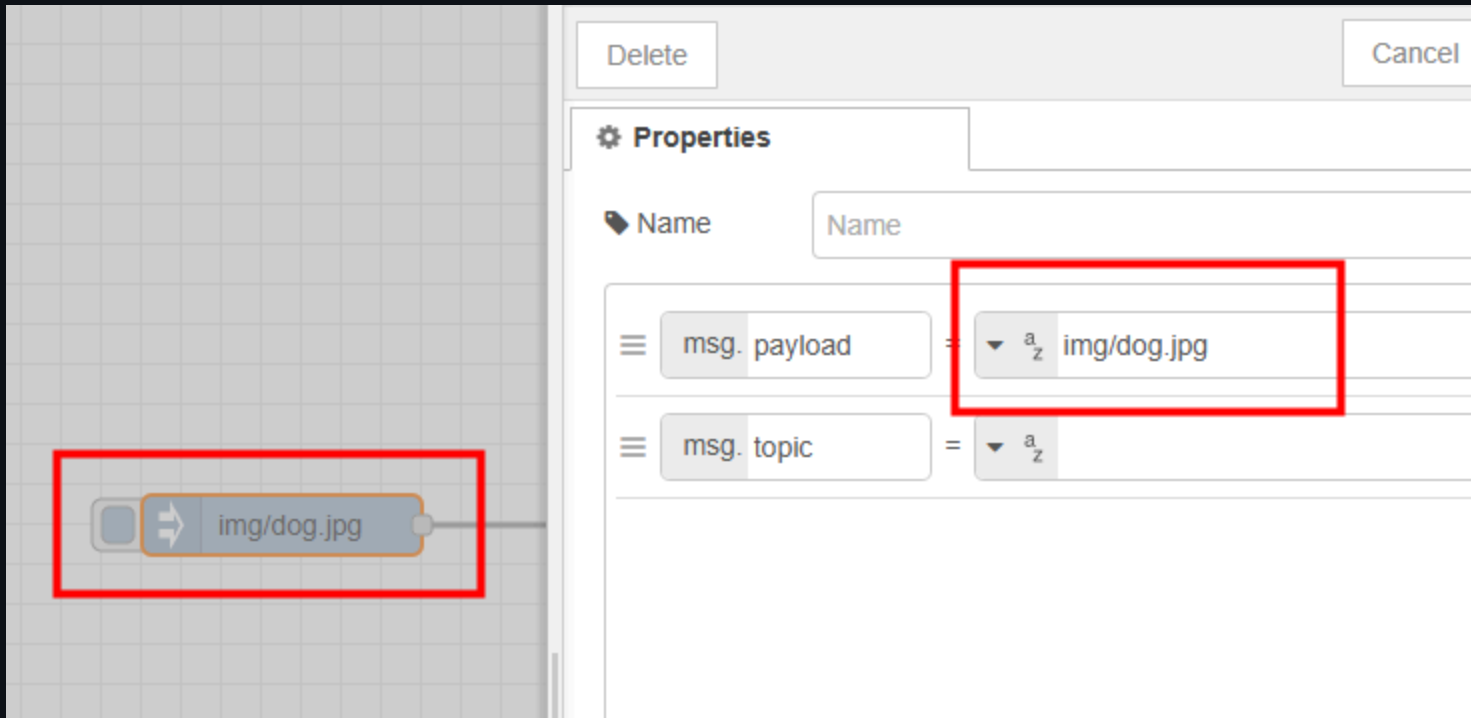
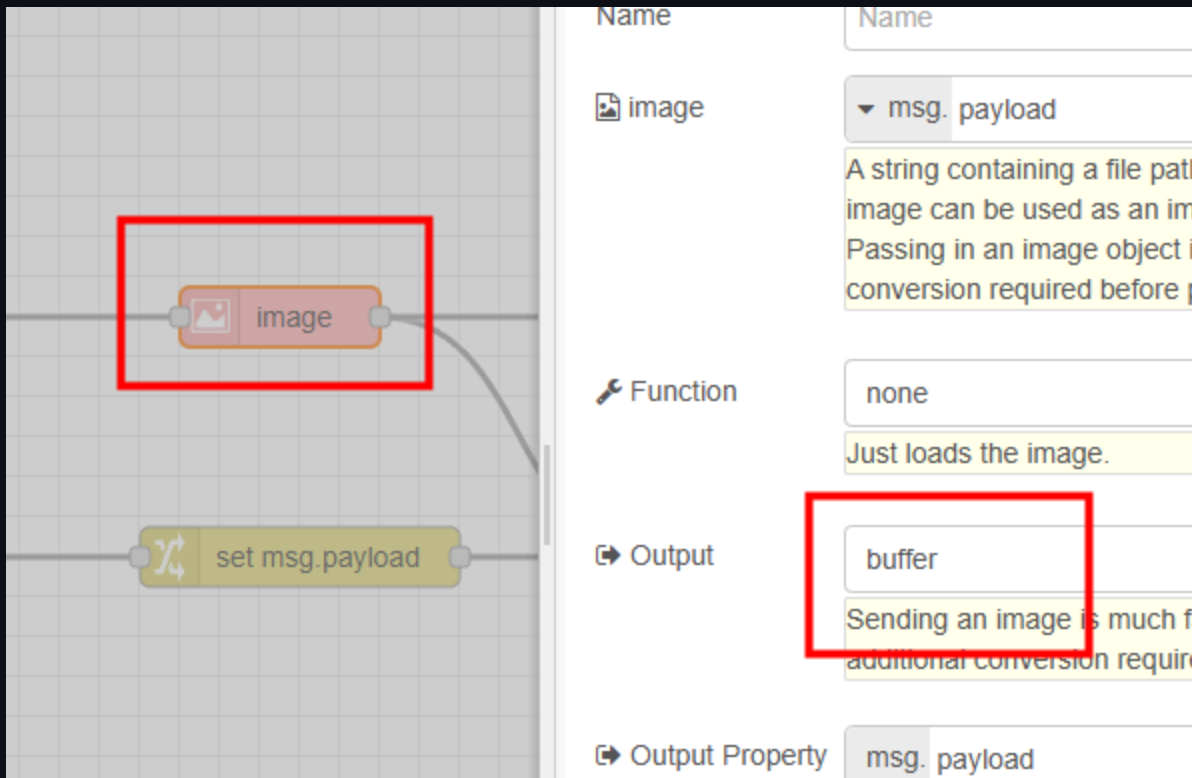


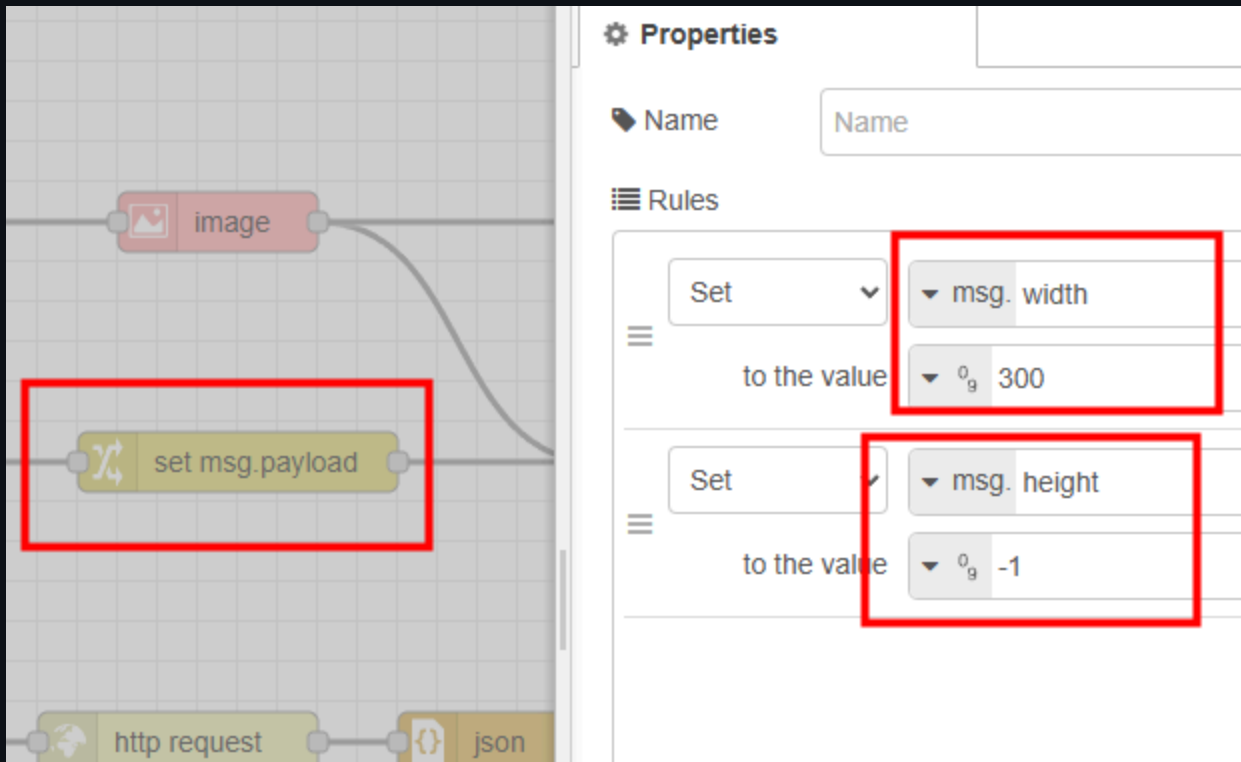
Image Node Configuration



The screenshot displays the Node-RED web interface. On the left, a workflow is visible on a grid background. It consists of two nodes: an 'image' node (orange with a picture icon) and a 'set msg.payload' node (yellow with a double-headed arrow icon). The 'image' node is highlighted with a red rectangle. A wire connects the output of the 'image' node to the input of the 'set msg.payload' node. On the right, the configuration panel for the selected 'image' node is shown. It has a 'Name' field with the value 'image'. Below this, there are three sections: 'Function' (set to 'none'), 'Output' (set to 'buffer'), and 'Output Property' (set to 'msg. payload'). The 'Output' section is highlighted with a red rectangle. The descriptions for the 'Function' and 'Output' sections are highlighted in yellow.

Property	Value
Name	image
Function	none Just loads the image.
Output	buffer Sending an image is much faster than sending a string. No additional conversion required.
Output Property	msg. payload

Change Node Configuration



The screenshot displays the Node-RED web interface. On the left, a workflow is visible with an 'image' node connected to a 'set msg.payload' node, which is highlighted with a red rectangle. Below it, an 'http request' node is connected to a 'json' node. On the right, the 'Properties' panel for the 'set msg.payload' node is shown. It includes a 'Name' field and a 'Rules' section. The 'Rules' section contains two entries, each highlighted with a red rectangle:

- The first rule is 'Set' with a dropdown menu set to 'msg. width' and a value of '300'.
- The second rule is 'Set' with a dropdown menu set to 'msg. height' and a value of '-1'.

Image Node Configuration (Resize)

The screenshot displays the Node-RED web interface. On the left, a workflow is visible with a 'resize' node (orange box with a red border) connected to a 'viewer' node. Below this, there are other nodes like 'mat Prediction Data', 'mat Count Data', 'mat Select-Class Data', and 'Show Image'. On the right, the configuration panel for the 'resize' node is open. It has four sections: 'Function' with a 'resize' dropdown, 'Output' with a 'buffer' dropdown, 'Output Property' with a 'msg. payload' dropdown, and two input fields for width and height. The width field is labeled '</> w' and the height field is labeled '</> h'. Both the width and height dropdowns are set to 'msg. width' and 'msg. height' respectively, and both are highlighted with red boxes. The dropdowns also show a description: '(Required) the width to resize the image ("Jimp.AUTO" or -1)' for width and '(Required) the height to resize the image ("Jimp.AUTO" or -1)' for height.

Function: **resize**

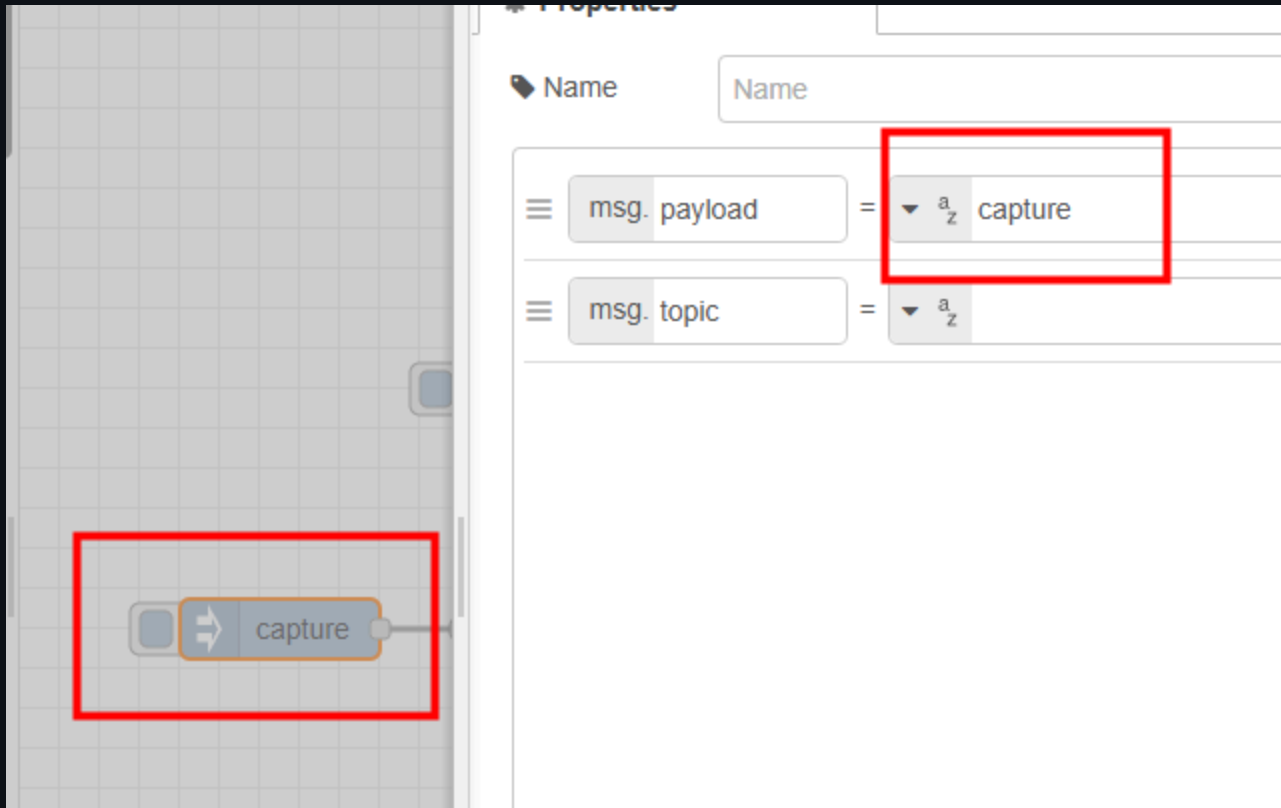
Output: **buffer**

Output Property: **msg. payload**

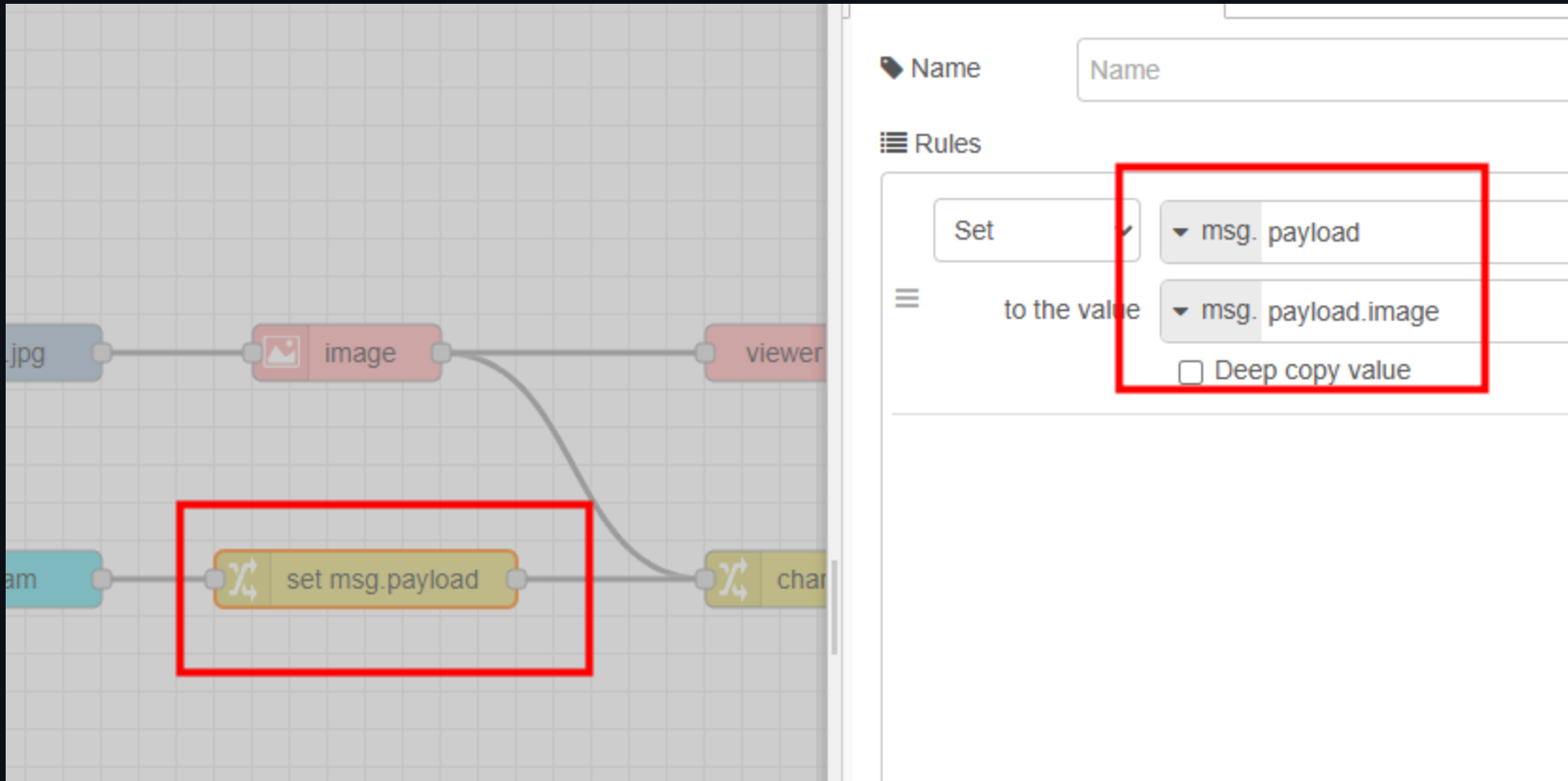
</> w: **msg. width**
(Required) the width to resize the image ("Jimp.AUTO" or -1)

</> h: **msg. height**
(Required) the height to resize the image ("Jimp.AUTO" or -1)

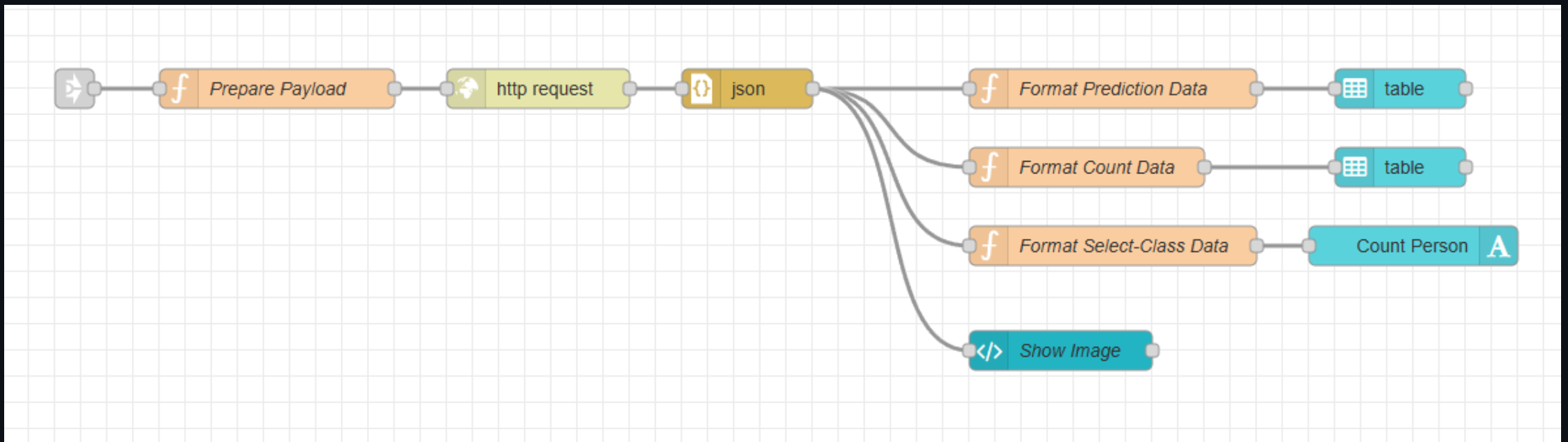
Inject Node Configuration (Capture)



Change Node Configuration (Webcam)



Object Detection Flow



Prepare Payload Function Node

```
msg.headers = {  
  "content-type": "multipart/form-data",  
};  
msg.payload = {  
  img: {  
    value: msg.payload,  
    options: {  
      filename: "a.jpg",  
      contentType: "image/jpeg",  
    },  
  },  
};  
msg.url = "http://localhost:3004/upload"; // ML server endpoint  
return msg;
```

HTTP Request Node Configuration

The screenshot displays the Node-RED web interface. On the left, a flow diagram is visible with several nodes. The 'http request' node is highlighted with a red rectangle. On the right, the 'Properties' sidebar is open, showing the configuration for the selected 'http request' node. The 'Method' is set to 'POST' and the 'URL' is set to 'https://'. Other options like 'Enable secure (SSL/TLS) connection', 'Use authentication', 'Enable connection keep-alive', 'Use proxy', 'Only send non-2xx responses to Catch node', and 'Disable strict HTTP parsing' are listed with checkboxes. The 'Return' dropdown is set to 'a UTF-8 string'.

Properties

- Method: POST
- URL: https://
- ☐ Enable secure (SSL/TLS) connection
- ☐ Use authentication
- ☐ Enable connection keep-alive
- ☐ Use proxy
- ☐ Only send non-2xx responses to Catch node
- ☐ Disable strict HTTP parsing
- Return: a UTF-8 string
- Headers:

Format Prediction Data Function Node

```
const predictions = msg.payload?.predictions;
if (!Array.isArray(predictions)) {
  msg.payload = [];
  return msg;
}

const dt = new Date();
const datestring = dt.toLocaleDateString();
const timestring = dt.toLocaleTimeString();
msg.payload = predictions.map((d) => ({
  Class: d["label"],
  Score: `${Math.round(d["score"] * 100)}%`,
  Location: JSON.stringify(d.box),
  Time: `📅 ${datestring} ⌚ ${timestring}`,
})));
return msg;
```

Table Node Configuration

The screenshot displays the configuration panel for a 'table' node in a data flow tool. On the left, a workflow diagram shows a 'Data' node connected to a 'table' node, which is highlighted with a red rectangle. Below it, another 'table' node is connected to a 'Count Person' node. The configuration panel on the right includes the following settings:

- Size:** auto
- Label:** optional label
- Class:** Optional CSS class name(s)
- Max Rows:** 0
- Action:** Replace (highlighted with a red rectangle)
- Breakpoint:** defaults: Mobile (< 576px)
- Interaction:** None
- Search:** ☒ Show

Format Count Data Function Node

```
const countsArr = msg.payload?.countsArr;
if (!Array.isArray(countsArr)) {
  msg.payload = [];
  return msg;
}

const dt = new Date();
const datestring = dt.toLocaleDateString();
const timestring = dt.toLocaleTimeString();
msg.payload = countsArr.map((d) => ({
  Class: d["class"],
  Count: d["count"],
  Time: `📅 ${datestring} ⌚ ${timestring}`,
})));
return msg;
```

Format Select-Class Data Function Node

```
const countsArr = msg.payload?.countsArr;
if (!Array.isArray(countsArr)) {
  msg.payload = [];
  return msg;
}

const dt = new Date();
const datestring = dt.toLocaleDateString();
const timestring = dt.toLocaleTimeString();
msg.payload = countsArr.map((d) => ({
  Class: d["class"],
  Count: d["count"],
  Time: `📅 ${datestring} ⌚ ${timestring}`,
})));
return msg;
```

Template Node Configuration

```
<template>
  <div>
    
  </div>
</template>

<script>
  export default {};
</script>
<style></style>
```